(a) A coding conversation: later inputs carry the earlier history

Illustrative example

Model call 1: input

H1

User: Add timeout=30 to connect().

Output: read_file("client.py")

Tool returns the source.

Model call 2: input

H1

User: Add timeout=30 to connect().

H2

Assistant: read_file("client.py")

Tool: def connect(host): ...

Output: apply_patch(...)

Tool confirms the edit.

Model call 3: input

H1

User: Add timeout=30 to connect().

H2

Assistant: read_file("client.py")

Tool: def connect(host): ...

H3

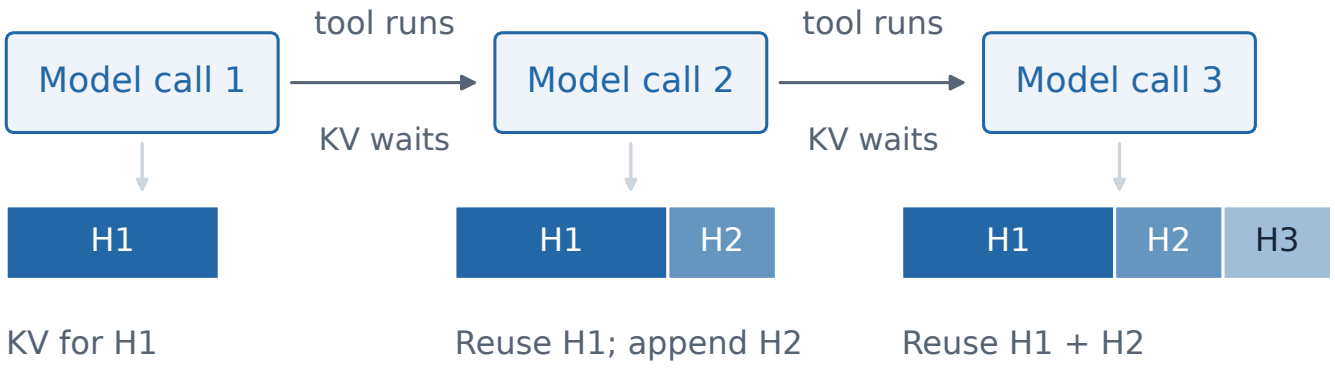
Assistant: apply_patch(...)

Tool: Timeout argument added.

Output: "Done; timeout added."

Same H label / color = identical message span

(b) One session: history maps to KV prefixes



Retain the prefix → save prefill when the session resumes

(c) Many sessions: finite capacity

Shared KV cache (full)

needs space

A / B / C: next demand unknown

Which idle KV should stay?

Keep occupy capacity during the wait

Evict recompute if the prefix is reused